

SSH Keys

In the physical world, we keep things secure by using locks that are opened by keys. In cyberspace, we secure things by **SSH keypairs** that work the same way. A keypair consists of two files:

In the physical world, we keep things secure by using locks that are opened by keys. In cyberspace, we secure things by **SSH keypairs** that work the same way. A keypair consists of two files:

1. **A public key** – This is like the lock on the box. You can share this lock with anyone because they can't open the box with just the lock.

In the physical world, we keep things secure by using locks that are opened by keys. In cyberspace, we secure things by **SSH keypairs** that work the same way. A keypair consists of two files:

1. **A public key** – This is like the lock on the box. You can share this lock with anyone because they can't open the box with just the lock.
2. **A private key** – This is your key that opens the lock. You keep it secret and safe.

What are SSH keys?

When working with GitHub (or any similar service), **SSH keys** work as follows:

What are SSH keys?

When working with GitHub (or any similar service), **SSH keys** work as follows:

- Your **public key** is uploaded to GitHub.

What are SSH keys?

When working with GitHub (or any similar service), **SSH keys** work as follows:

- Your **public key** is uploaded to GitHub.
- Your **private key** stays on your computer or container.

What are SSH keys?

When working with GitHub (or any similar service), **SSH keys** work as follows:

- Your **public key** is uploaded to GitHub.
- Your **private key** stays on your computer or container.
- When you try to connect to GitHub using your account, it will present you with a challenge that is encrypted using the public key. Your computer will then decrypt this using the private key, and GitHub will check if the challenge matches before you're allowed in.

What are SSH keys?

When working with GitHub (or any similar service), **SSH keys** work as follows:

- Your **public key** is uploaded to GitHub.
- Your **private key** stays on your computer or container.
- When you try to connect to GitHub using your account, it will present you with a challenge that is encrypted using the public key. Your computer will then decrypt this using the private key, and GitHub will check if the challenge matches before you're allowed in.

What are SSH keys?

When working with GitHub (or any similar service), **SSH keys** work as follows:

- Your **public key** is uploaded to GitHub.
- Your **private key** stays on your computer or container.
- When you try to connect to GitHub using your account, it will present you with a challenge that is encrypted using the public key. Your computer will then decrypt this using the private key, and GitHub will check if the challenge matches before you're allowed in.

SSH keys are both secure and convenient – no need to enter a password every time you connect.

Why use SSH keys with Docker and GitHub?

To access GitHub from inside a **Docker container** you need to set up SSH keys so the container can communicate securely with GitHub.

Why use SSH keys with Docker and GitHub?

To access GitHub from inside a **Docker container** you need to set up SSH keys so the container can communicate securely with GitHub.

By storing your keys in a **persistent volume**, you can avoid regenerating keys every time you start the container.

Store your keypair in a persistent volume

When creating your container, make sure to add a volume – a shared folder between the container and your computer. In our original docker setup, we called it **/workspace**. This directory can store SSH keys persistently.

Store your keypair in a persistent volume

When creating your container, make sure to add a volume – a shared folder between the container and your computer. In our original docker setup, we called it **/workspace**. This directory can store SSH keys persistently.

Let's spin up our container and add a subdirectory:

Store your keypair in a persistent volume

When creating your container, make sure to add a volume – a shared folder between the container and your computer. In our original docker setup, we called it **/workspace**. This directory can store SSH keys persistently.

Let's spin up our container and add a subdirectory:

```
mkdir -p /workspace/secrets
```

Store your keypair in a persistent volume

Inside the container, generate a new SSH keypair:

Store your keypair in a persistent volume

Inside the container, generate a new SSH keypair:

```
ssh-keygen -t ed25519 -C "joachim@class-docker"
```

Store your keypair in a persistent volume

Inside the container, generate a new SSH keypair:

```
ssh-keygen -t ed25519 -C "joachim@class-docker"
```

- `-t ed25519` specifies the key type.

Store your keypair in a persistent volume

Inside the container, generate a new SSH keypair:

```
ssh-keygen -t ed25519 -C "joachim@class-docker"
```

- `-t ed25519` specifies the key type.
- `-C "joachim@class-docker"` adds a comment for identification. You can write anything here.

Store your keypair in a persistent volume

The command creates two files in `/root/.ssh/`:

Store your keypair in a persistent volume

The command creates two files in `/root/.ssh/`:

- `id_ed25519` (private key)

Store your keypair in a persistent volume

The command creates two files in `/root/.ssh/`:

- `id_ed25519` (private key)
- `id_ed25519.pub` (public key)

Store your keypair in a persistent volume

The command creates two files in `/root/.ssh/`:

- `id_ed25519` (private key)
- `id_ed25519.pub` (public key)

Store your keypair in a persistent volume

The command creates two files in `/root/.ssh/`:

- `id_ed25519` (private key)
- `id_ed25519.pub` (public key)

Immediately copy these to the mounted volume:

Store your keypair in a persistent volume

The command creates two files in `/root/.ssh/`:

- `id_ed25519` (private key)
- `id_ed25519.pub` (public key)

Immediately copy these to the mounted volume:

```
cp /root/.ssh/id* /workspace/secrets/
```

Store your keypair in a persistent volume

The command creates two files in `/root/.ssh/`:

- `id_ed25519` (private key)
- `id_ed25519.pub` (public key)

Immediately copy these to the mounted volume:

```
cp /root/.ssh/id* /workspace/secrets/
```

Print the public key:

Store your keypair in a persistent volume

The command creates two files in `/root/.ssh/`:

- `id_ed25519` (private key)
- `id_ed25519.pub` (public key)

Immediately copy these to the mounted volume:

```
cp /root/.ssh/id* /workspace/secrets/
```

Print the public key:

```
cat /workspace/secrets/id_ed25519.pub
```

Store your keypair in a persistent volume

The command creates two files in `/root/.ssh/`:

- `id_ed25519` (private key)
- `id_ed25519.pub` (public key)

Immediately copy these to the mounted volume:

```
cp /root/.ssh/id* /workspace/secrets/
```

Print the public key:

```
cat /workspace/secrets/id_ed25519.pub
```

Copy the output.

- Go to GitHub SSH settings.

Add the public key to GitHub

- Go to GitHub SSH settings.
- Click **New SSH Key**.

Add the public key to GitHub

- Go to GitHub SSH settings.
- Click **New SSH Key**.
- Paste the public key you copied and save it.

Add the public key to GitHub

- Go to GitHub SSH settings.
- Click **New SSH Key**.
- Paste the public key you copied and save it.
 - Make sure you don't copy line breaks.

Test the connection

Inside the container, test the SSH connection:

Test the connection

Inside the container, test the SSH connection:

```
ssh -T git@github.com
```

Test the connection

Inside the container, test the SSH connection:

```
ssh -T git@github.com
```

First you'll see a message about trusting the server you're connecting to.

Test the connection

Inside the container, test the SSH connection:

```
ssh -T git@github.com
```

First you'll see a message about trusting the server you're connecting to.

Then, if everything works, you'll see a message like:

Test the connection

Inside the container, test the SSH connection:

```
ssh -T git@github.com
```

First you'll see a message about trusting the server you're connecting to.

Then, if everything works, you'll see a message like:

```
Hi username! You've successfully authenticated.
```

By generating SSH keys inside the Docker container and saving them in a persistent volume:

By generating SSH keys inside the Docker container and saving them in a persistent volume:

- You've enabled secure access to GitHub without needing passwords.

By generating SSH keys inside the Docker container and saving them in a persistent volume:

- You've enabled secure access to GitHub without needing passwords.
- The keys are saved outside the container, so they remain available even if the container is restarted or recreated.

By generating SSH keys inside the Docker container and saving them in a persistent volume:

- You've enabled secure access to GitHub without needing passwords.
- The keys are saved outside the container, so they remain available even if the container is restarted or recreated.
- You're now set up to clone, pull, and push repositories from inside the Docker container.